By, Nabajyoti Medhi

IPv4 planning, then emulate *it.*

TEN THINGS YOU WILL DO

01–05 Plan an IPv4 scheme: address, mask, range, broadcast, usable hosts.

— Subnet by hand, allocate VLSM blocks, finish with a one-page cheatsheet.

06–10 · EMULATE

— Bring up Mininet, build custom topologies, route, measure, attach a remote controller.

FOUR OCTETS, 32 BITS

octet 1 · octet 2 · octet 3 · octet 4

Then a host kernel runs the same logic—Linux, namespaces, OVS, OpenFlow.

Plan paper-side. Then bring it up on a Linux process tree.

32 bits, four octets, one mask. 1s for the network, 0s for the host.

192	168	10	77
octet 1	octet 2	octet 3	octet 4

BINARY

11000000
10101000
00001010
01001101

AND MASK /26

192.168.10.64

NETWORK ADDRESS

CLASSES · HISTORICAL

A · /8	B · /16	C · /24	D · multicast	E · reserved
0–127 · 16.7M hosts · 127/8 is loopback	128–191 · 65,534 hosts	192–223 · 254 hosts · lab default	224–239 · 224.0.0.0/4	240–255 · 240/4 not assignable

AND · 1·1 = 1

Bit-by-bit. Mask zeroes the host field, keeps the prefix.

USABLE HOSTS

$2^h - 2$. Network and broadcast are reserved.

INCREMENT

256 − last non-255 octet. Block size for that subnet.

EXCEPTIONS

/31 = 2 hosts, no broadcast (RFC 3021). /32 = host route.

/24 into four /26s, named out loud.

borrow **2 bits** · **4 subnets** · host 6 bits · **62 usable** · increment **256 − 192 = 64**

N	NETWORK / PREFIX	USABLE RANGE	BROADCAST	USABLE
01	192.168.10.0/26	.1 — .62	.63	62
02	192.168.10.64/26	.65 — .126	.127	62
03	192.168.10.128/26	.129 — .190	.191	62
04	192.168.10.192/26	.193 — .254	.255	62

CIDR AGGREGATION

198.51.100.0/22

1024 addresses · 1 route

Aggregates .0/24 + .1/24 + .2/24 + .3/24 — must start on a /22 boundary.

FOUR · /24 BLOCKS

198.51.100.0/.1/.2/.3 · /24



LONGEST-MATCH WINS

A /24 inside the /22 overrides the aggregate when both match.

Per RFC 4632 §5.1.

WHEN TO USE

Fixed-mask subnetting works when segments are roughly equal. Switch to VLSM (slide 04) when they are not.

Right-size every block. Walk the 5 steps.

STEP 01
List
Each segment's host count, with growth margin.

STEP 02
Sort
Descending. The largest first.

STEP 03
Choose
Smallest prefix whose 2^h-2 fits.

STEP 04
Allocate
Aligned on the next prefix boundary.

STEP 05
Verify
No overlap. Document ranges & broadcasts.

WORKED LAB · 192.168.10.0/24 · SEVEN SEGMENTS

SEGMENT	USABLE	NEEDS PREFIX	NETWORK
HQ-DC (servers)	100	/25	192.168.10.0
OFFICE-A	126	40	/26	192.168.10.128
VOICE-VLAN	62	12	/28	192.168.10.192
NETMGMT	14	6	/29	192.168.10.208
+ 3 WAN /30 links · reserved /27	6	—	/27	.236 onwards
	26			

USED 222 / 254

87% utilization, 32 reserved.

Five masks (.25, .26, .28, .29, .30 × 3). Fixed-mask on a /24 wastes ~56%.

Two problems. One cheatsheet.

PROBLEM A · FIXED-LENGTH ~5 MIN

Subnet `192.168.50.0/24` into `/27`s.

State mask, subnets, usable hosts each, then network/broadcast/range for the first two.

ANSWER PREVIEW

/27 = 255.255.255.224 · 2^3 = 8 subnets · 2^5−2 = 30 hosts · increment 32.

PROBLEM B · VLSM ~7 MIN

Allocate from `10.10.0.0/23`: 100 + 50 + 20 hosts.

Largest first, smallest prefix whose usable count fits each, document ranges.

ANSWER PREVIEW

10.10.0.0/25 (126) · 10.10.0.128/26 (62) · 10.10.0.192/27 (30). Sum 224, fits /23.

FOUR FAILURES YOU WILL MAKE ONCE

Overlapping subnets.
Allocate aligned & non-overlapping.

Parent too small.
Sum requirements must fit before splitting.

Broadcast is not a host.
Usable = total − 2. Watch last IP in range.

Non-contiguous mask.
CIDR rejects. Ones lead, zeros trail.

CHEATSHEET · KEEP THIS PAGE

NETWORK & HOST

```
prefix bits = N
host bits  = 32 − N
total = 2^h
usable = 2^h − 2
```

SUBNETS / INCREMENT

```
borrowed bits = s
subnets = 2^s
increment =
    256 − last mask octet
```

FIND BOUNDARIES

```
IP AND mask → network
broadcast = net + size − 1
first host = net + 1
last host  = bcast − 1
```

EXCEPTIONS

```
/31 = 2 hosts, no broadcast —
   RFC 3021, point-to-point
/32 = one address, host route
   loopback / /32 route
```

A network, in a process tree.

Stanford-originated, community-maintained. Real Linux kernel, real OVS, real namespaces. Emulation — not simulation, not a fleet of VMs.

WHAT & WHERE

Hosts are Linux processes in their own network namespaces. Links are virtual Ethernet. Switches commonly run on *Open vSwitch*; the controller is OpenFlow.

Trade-off: lightweight and fast, but bounded by the host's CPU and Linux. Not a substitute for a production benchmark.

INSTALL · UBUNTU LTS

```
# package route — check version after install
sudo apt update
sudo apt install mininet
mn --version

# from source — Python 3 install
git clone https://github.com/mininet/mininet
sudo PYTHON=python3 util/install.sh -nfv
```

DEFAULT TOPOLOGY · sudo mn



Two hosts, one switch, an in-process OpenFlow reference controller. The CLI drops you in at `mininet>`.

CLI · AT mininet>

```
nodes      # list nodes
net        # links & interfaces
dump       # per-node info
pingall    # all-pairs probe
h1 ping -c 1 h2
link s1 h1 down  # failure
link s1 h1 up    # restore
iperf      # bandwidth
exit       # tear down & clean
```

Hosts live in their own namespaces; the switch commonly sits in the root namespace. `exit` destroys the current emulation; `sudo mn -c` cleans state if a crash left something behind.

Four ways to shape the lab.

SINGLE SWITCH · N HOSTS




```
sudo mn --topo single,4
```

One switch, every host on the same L2 segment.

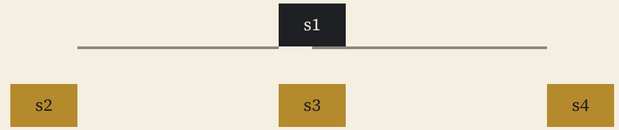
LINEAR · k SWITCHES, ONE HOST EACH




```
sudo mn --topo linear,4
```

LinearTopo(k=4, n=1) — chain of switches.

TREE · depth 2 · fanout 3




```
sudo mn --topo tree,2,3
```

1 root + 3 inner + 9 leaf hosts under depth=2, fanout=3.

CUSTOM TOPO · PYTHON

labtopo.py

```
#!/usr/bin/env python3
from mininet.topo  import Topo
from mininet.net   import Mininet
from mininet.cli   import CLI
class LabTopo(Topo):
    def build(self):
        h1 = self.addHost('h1', ip='10.0.0.11/24')
        h2 = self.addHost('h2', ip='10.0.0.12/24')
        s1 = self.addSwitch('s1')
        self.addLink(h1, s1)
        self.addLink(h2, s1)
topos = {'lab': LabTopo}
net = Mininet(topo=LabTopo())
net.start(); CLI(net); net.stop()
```

RUN IT

```
# direct run
sudo python3 labtopo.py

# or via mn --custom
sudo mn --custom ./labtopo.py \
  --topo lab
```

At the CLI: `dump`, `pingall`, `h1 arping h2`.

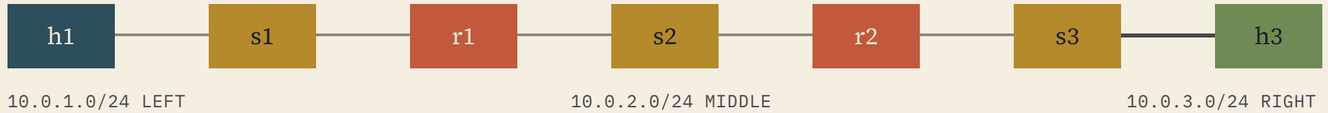
Make hosts become routers, then route.

ADDRESS PLAN

```
10.0.1.0/24  left LAN
    h1 = 10.0.1.10
10.0.2.0/24  middle · r1-r2
10.0.3.0/24  right LAN
    h3 = 10.0.3.10

r1 · 10.0.1.1 / 10.0.2.1
r2 · 10.0.2.2 / 10.0.3.1
```

TOPOLOGY SKETCH



Routers are `Node` subclasses with `inNamespace=True`; forwarding is enabled with `sysctl -w net.ipv4.ip_forward=1`.

PYTHON · routed.py

routed.py

```
from mininet.topo  import Topo
from mininet.net   import Mininet
from mininet.node import Node
from mininet.cli   import CLI
class LinuxRouter(Node):
    def config(self, **p):
        super().config(**p)
        self.cmd('sysctl -w net.ipv4.ip_forward=1')
class RoutedTopo(Topo):
    def build(self):
        r1 = self.addNode('r1', cls=LinuxRouter, inNamespace=True)
        r2 = self.addNode('r2', cls=LinuxRouter, inNamespace=True)
        for n in ('s1','s2','s3'): self.addSwitch(n)
        self.addLink('s1', r1, intfName2='r1-eth0', params2={'ip':'10.0.1.1/24'})
        self.addLink('s2', r1, intfName2='r1-eth1', params2={'ip':'10.0.2.1/24'})
        self.addLink('s2', r2, intfName2='r2-eth0', params2={'ip':'10.0.2.2/24'})
        self.addLink('s3', r2, intfName2='r2-eth1', params2={'ip':'10.0.3.1/24'})
        self.addHost('h1', ip='10.0.1.10/24', defaultRoute='via 10.0.1.1')
        self.addHost('h3', ip='10.0.3.10/24', defaultRoute='via 10.0.3.1')
        self.addLink('h1', s1); self.addLink('h3', s3)
net = Mininet(topo=RoutedTopo())
net.start(); CLI(net); net.stop()
```

VERIFY

```
sudo python3 routed.py

mininet> r1 ip route \
  add 10.0.3.0/24 via 10.0.2.2
mininet> r2 ip route \
  add 10.0.1.0/24 via 10.0.2.1
mininet> pingall
mininet> h1 ping -c 3 h3
```

Shape the link, then measure.

Bandwidth, delay, and loss are emulated per-link through Linux traffic control. Run the same probe, change one parameter, compare.

IPERF · THROUGHPUT

```
# inside the CLI
mininet> iperf

# run-once flag
sudo mn --test iperf
```

Server on h1, client on h2. One run is not a capacity number.

PING · LATENCY & LOSS

```
# send 10 probes
mininet> h1 ping -c 10 h2
```

RTT includes delay in both directions plus host scheduling.

PARAMETER CARDS

bw= rate cap in Mbit/s
delay= per-link one-way delay
loss= percentage loss

Change one parameter at a time, then re-measure.

CLI · TC LINK

```
# 10 Mbit/s, 5 ms one-way, 1% loss
sudo mn --link tc,bw=10,delay=5ms,loss=1

# then probe inside the CLI
mininet> iperf
mininet> h1 ping -c 10 h2
```

Drop a parameter at a time, record what actually changes.

TCLINK · PYTHON EQUIVALENT

```
from mininet.link import TCLink
from mininet.topo  import Topo
from mininet.net   import Mininet

class TcTopo(Topo):
    def build(self):
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        s1 = self.addSwitch('s1')
        self.addLink(h1, s1, bw=10, delay='5ms', loss=1)
        self.addLink(h2, s1, bw=10, delay='5ms', loss=1)

net = Mininet(topo=TcTopo(), link=TCLink)
net.start(); CLI(net); net.stop()
```

Decouple the control plane.

Mininet switches talk OpenFlow. Default controller is in-process; real lab work attaches a separate controller (Ryu, POX, ONOS).

REFERENCE CONTROLLER

```
sudo mn
# in-process, learning-switch behavior
```

Good for the default lab. Behavior is hard-coded in Mininet itself.

REMOTE CONTROLLER

```
sudo mn \
  --controller remote,ip=127.0.0.1,port=6653
```

Switch learns whatever the controller programs. Match the controller's listener port.

PORT NOTE

Modern OpenFlow port is **6653** (IANA). The historical 6633 lives on in some POX examples — match your controller's actual listener.

RYU WALKTHROUGH

TERMINAL 1 · CONTROLLER

```
# Learning-switch app, OpenFlow 1.3
ryu-manager ryu.app.simple_switch_13
```

Visible activity in this window will show packet-in events when hosts ARP or forward unknown flows.

TERMINAL 2 · MININET

```
sudo mn --topo single,2 \
  --controller remote, \
  ip=127.0.0.1,port=6653
```

In the CLI: `h1 ping -c 1 h2`. Watch Terminal 1 light up.

See what's on the wire. Then name it.

The `ip` command from `iproute2` replaces `ifconfig`. Start here before touching routes or neighbours.

INSPECT

```
ip link       # link state up/down, mtu, qdisc
ip -s link    # counters: rx/tx errors, drops
ip addr     # addresses + scope on each iface
ip -br addr   # brief — name + addr only
ip -s addr    # lifetime, scope, dynamic v4/v6
ip route      # see slide 12 for routing
```

CONFIGURE

```
# add an address
ip addr add 10.0.1.10/24 dev eth0
# delete one
ip addr del 10.0.1.10/24 dev eth0
# bring up / bring down
ip link set eth0 up
ip link set eth0 down
# flush before reconfiguring
ip addr flush dev eth0
```

SAMPLE OUTPUT · ip -br addr

```
$ ip -br addr
lo         UNKNOWN  127.0.0.1/8 ::1/128
eth0       UP       10.0.1.10/24 fe80::1/64
eth1       UP       10.0.2.10/24 fe80::2/64
docker0    DOWN      172.17.0.1/16
```

UP = link up. The address column reports prefix length. Flags like qlen / scope live in `ip -d addr`.

PERSISTENCE — WATCH OUT

`ip addr add` / `ip link set` changes are not durable across reboot by themselves.

On Ubuntu Server they live in `/etc/netplan/*.yaml`. On RHEL family they live in `/etc/NetworkManager/system-connections/` or `/etc/sysconfig/network-scripts/`.

Slide 15 covers system-wide tunables via `sysctl`.

Read the table. Push routes into it.

Forwarding is on when `/proc/sys/net/ipv4/ip_forward` is 1. `ip route` commands write and read the kernel FIB.

READ THE TABLE

```
ip route     # full table
ip -br route  # brief summary
ip route show table main
ip route show table local
ip route get 8.8.8.8
ip route show cache
# rule-based routing
ip rule list
```

WRITE INTO THE TABLE

```
# static route via gateway
ip route add 10.0.3.0/24 via 10.0.2.2
# default route
ip route add default via 10.0.1.1
# on-link for the iface subnet
ip route add 10.0.1.0/24 dev eth0
# remove
ip route del 10.0.3.0/24
ip route flush cache
```

FORWARDING · POLICY ROUTING

```
cat /proc/sys/net/ipv4/ip_forward
# turn it on (must be 1 to route)
sysctl -w net.ipv4.ip_forward=1
# add a rule and a table
echo 200 mytable >> /etc/iproute2/rt_tables
ip rule add from 192.168.5.0/24 table mytable
ip route add default via 192.168.5.1 table mytable
```

PERSISTENCE · NETPLAN EXAMPLE

```
# /etc/netplan/50-routes.yaml
network:
  version: 2
  ethernets:
    eth0:
      routes:
       - to: 10.0.3.0/24
         via: 10.0.2.2
```

The neighbour table ties L3 to L2.

ARP for IPv4, ND for IPv6. `ip neigh` replaces `arp`. Each entry shows state and age in seconds.

READ · ADD · DELETE

```
ip neigh           # full table (IPv4 + IPv6)
ip -br neigh      # brief — NAME MAC DEV
ip neigh show dev eth0
# static entry (override)
ip neigh add 10.0.1.5 lladdr 00:11:22:33:44:55 dev eth0
# delete one / flush all on iface
ip neigh del 10.0.1.5 dev eth0
ip neigh flush dev eth0
```

SAMPLE OUTPUT

```
$ ip -br neigh
10.0.1.1 dev eth0 lladdr aa:bb:cc:dd:ee:ff REACHABLE
10.0.1.7 dev eth0 lladdr 00:11:22:33:44:55 STALE
10.0.1.99 dev eth0   FAILED
```

States include REACHABLE, STALE, DELAY, PROBE, FAILED, NOARP, PERMANENT.

PROBE · arping

```
# trigger a probe (force resolution)
ip neigh replace 10.0.1.5 lladdr aa:bb:cc:dd:ee:ff dev eth0
# classic arp tool still works
arp -n
arp -i eth0 -s 10.0.1.5 00:11:22:33:44:55
```

For an unsolicited ARP, use `arping -U -c 1 -I eth0 10.0.1.5` — newer kernels moved this to `ip`.

WHEN ARP BREAKS

A **STALE** neighbour is fine. A **FAILED** one breaks the local subnet.

```
# common culprits
- VLAN mismatch on the switch port
- Promiscuous or 802.1X block
- MAC learning disabled
- ARP proxy misconfigured
```

Probe the path. Capture bytes.

Reachability, routing trace, listening sockets, packet capture — four tools cover ninety percent of on-host diagnosis.

REACHABILITY · PATH

```
# 3 ICMP probes, 1-second interval
ping -c 3 8.8.8.8
# stop after first reply (liveness)
ping -c 1 -W 1 10.0.1.1
# path discovery, UDP style (no root)
tracepath 8.8.8.8
# continuous trace, refreshed stats
mtr -rwc 20 8.8.8.8
# which gateway + interface chosen?
ip route get 10.0.1.10
```

SOCKETS · LISTENERS

```
# listening TCP + UDP, with PID + inode
ss -tulpn
# established TCP only
ss -tan state established
# count of TIME_WAIT per remote
ss -tan state time-wait | wc -l
# which process holds port 8080?
ss -tlnp 'sport = :8080'
# host's external view of its own IPs
hostname -I    hostname -f
```

CAPTURE · tcpdump + BPF

```
# 50 packets on eth0, no name lookup     # host-to-192.168 only
tcpdump -i eth0 -n -c 50    tcpdump -i eth0 -n host 192.168.10.5
# TCP port 80, save to pcap   # ICMP echo only
tcpdump -i any -w /tmp/cap.pcap tcp port 80  tcpdump -i eth0 -n icmp
# read back later                        # no DNS, no port-name lookups
tcpdump -nn -r /tmp/cap.pcap    add -nn almost everywhere
```

NAME LOOKUP

```
getent hosts archive.ubuntu.com
# bypass /etc/hosts confusion
dig +short archive.ubuntu.com A
# trace resolver path
dig +trace example.com
# reverse the IP back to name
dig -x 8.8.8.8
```

Survive the reboot. Tune the kernel.

`ip` commands edit kernel state only. To persist, write a config — or change `sysctl` tunables.

UBUNTU · /etc/netplan/*.yaml

```
# /etc/netplan/50-eth0.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      addresses: [10.0.1.10/24]
      routes:
       - to: 10.0.3.0/24
         via: 10.0.2.2
sudo netplan apply
```

RHEL · FEDORA · nmcli

```
# show all connections
nmcli connection show
# add a static IPv4 + route + DNS
sudo nmcli con add type ethernet ifname eth0 \
    con-name lab ipv4.method manual \
    ipv4.addresses 10.0.1.10/24 \
    ipv4.gateway 10.0.1.1 ipv4.dns 1.1.1.1
sudo nmcli con up lab
```

SYSCTL TUNABLES · /etc/sysctl.d/*.conf

```
# enable v4 forwarding on a router
net.ipv4.ip_forward = 1
# reverse-path filter (anti-spoof)
net.ipv4.conf.all.rp_filter = 1
# don't accept source routes
net.ipv4.conf.all.accept_source_route = 0
# ignore ICMP redirects
net.ipv4.conf.all.accept_redirects = 0
```

```
sudo sysctl --system   # reload all .conf
sudo sysctl -w net.ipv4.ip_forward=1
sysctl net.ipv4.ip_forward  # read current
# safety: never set rp_filter=0
# on a public-facing host
```

RECAP · FIVE LINUX SLIDES 11–15

11 · ip link · ip addr
12 · ip route · ip rule
13 · ip neigh · arping
14 · ping · mtr · ss · tcpdump · dig
15 · netplan · nmcli · sysctl

Read first, write second, persist third. Verify after every change with `ip route get` and `ping -c 1`.